

Web Attacks

NECKER

December 2, 2025

Contents

1	Document Object Model (DOM)	2
1.1	Fundamental DOM Objects	2
1.2	Relevant Attributes of the DOM and HTTP	2
2	HTTP Cookies	2
2.1	Operating Mechanism	2
2.2	Cookie Types	2
2.3	Cookie Attributes	3
2.4	JavaScript Access to Cookies	3
3	Same-Origin Policy (SOP)	3
3.1	Same-Origin Policy for the DOM (DOM SOP)	3
3.1.1	Origin Matching Criteria	3
3.1.2	Origin Matching Example	3
3.1.3	SOP Relaxation (<code>document.domain</code>)	4
3.2	Same-Origin Policy for Cookies	4
3.3	SOP and AJAX	4
4	Authentication Cookies, Malicious Scripts, and CSRF	5
4.1	Session ID and Cookie Theft	5
4.1.1	Security Risks (Cookie Theft)	5
4.1.2	Cookie Protection	5
4.2	Cross-Site Request Forgery (CSRF)	5
4.2.1	CSRF Attack Vectors	5
4.2.2	CSRF Mitigation	5
5	Cross-Site Scripting (XSS)	6
5.0.1	Potential Impact of XSS	6
5.1	Types of XSS Attacks	6
5.1.1	Stored XSS (Persistent)	6
5.1.2	Reflected XSS (Non-Persistent)	6
5.1.3	DOM-based XSS	6
5.2	XSS Mitigation and Input Sanitization	7
6	SQL Injection	7
6.1	Attack Mechanism	7
6.2	SQL Injection Mitigation	7

1 Document Object Model (DOM)

The **Document Object Model (DOM)** is the hierarchical internal representation of an HTML or XML document. It serves as an **API (Application Programming Interface)** that allows scripting languages (primarily JavaScript) to access and manipulate the structure, style, and content of the document (it is created from the **HTML** and resides in the browser's memory).

1.1 Fundamental DOM Objects

- **Window Object:** Represents the browser window (or a frame) in which the document is displayed. It is the root object.
- **Document Object:** It is a property of the window object (`window.document`). It forms the root of the hierarchical structure representing the content of the HTML document.
- **Location Object:** It is a property of the window object (`window.location`) and contains the components of the URL of the associated document.

1.2 Relevant Attributes of the DOM and HTTP

Table 1: DOM Attributes and HTTP Correlates

Attribute	Notes
Location (HTTP header)	Sent by the server for URL redirection.
Domain (HTTP cookie attribute)	Allows the origin server to broaden the cookie's scope.
<code>document.domain</code>	Hostname from which the document was loaded.
<code>window.location.href</code>	URL of the requested document.
<code>document.URL</code> (read-only)	URL of the loaded document.

2 HTTP Cookies

HTTP is a **stateless** protocol. **HTTP Cookies** are a **work-around** mechanism to maintain state (e.g., language preferences, shopping carts) between successive requests.

2.1 Operating Mechanism

1. **Setting:** The server sends one or more limited-size data strings (the cookie) to the client (browser) via the **Set-Cookie** response header.
2. **Storage:** The client stores the cookie.
3. **Return:** The client includes the cookie in the subsequent HTTP request sent to the **same site or pages** (depending on the scope) using the **Cookie** request header.

Cookies are "name=value" pairs followed by zero or more attributes.

2.2 Cookie Types

- **Session Cookies:** Short-lived cookies, stored in the browser's volatile memory and deleted when the window is closed.
- **Persistent Cookies:** They have an extended lifespan defined by the **Max-Age** or **Expires** attributes and are retained even after the browser is closed.

2.3 Cookie Attributes

Optional attributes control the lifespan, scope, and security restrictions of a cookie:

- **Max-Age=value in seconds** or **Expires=date**: Defines the maximum lifespan of the cookie (persistent cookie).
- **Domain=value**: Broadens the cookie's scope to a suffix domain. If set to `myhost.com`, the cookie is available to `myhost.com` and all of its subdomains. It cannot be set for a TLD (e.g., `.com`) for security reasons.
- **Path=value**: Controls which file system paths (pages) on the origin server a cookie is returned to. The default value is the directory of the requested resource.
- **Secure**: If present, the client must send the cookie **only** over encrypted connections (HTTPS/TLS).
- **HttpOnly**: If present, access to the cookie is permitted **only** via HTTP and denied to the DOM API (e.g., JavaScript) for security reasons (XSS mitigation).

2.4 JavaScript Access to Cookies

The DOM API `document.cookie` allows reading and writing access to all cookies for the current document. This access is denied if the `HttpOnly` attribute is set.

3 Same-Origin Policy (SOP)

The **Same-Origin Policy (SOP)** is an access control and isolation principle designed to prevent a document or script loaded from one source (**origin**) from interfering with (accessing or manipulating) resources originating from another origin.

3.1 Same-Origin Policy for the DOM (DOM SOP)

The fundamental rules for HTML documents and scripts are:

1. A base document has an origin defined by the URI that retrieved it.
2. Scripts and images inherit the origin of the HTML document that loaded them.
3. Scripts can access content (and DOM properties) whose assigned origin **matches** their own.

3.1.1 Origin Matching Criteria

Two origins are considered **identical** only if they have an identical **origin triplet** (**scheme**, **host**, **port**).

- **Scheme**: Protocol (e.g., `http`, `https`).
- **Host**: Hostname with the fully qualified domain name (FQDN). Subdomains are distinct origins.
- **Port**: Explicit port or the default port of the protocol (e.g., 80 for HTTP, 443 for HTTPS).

3.1.2 Origin Matching Example

- **Match**: `http://site.com/file1` and `http://site.com/file2` (same triplet).
- **Mismatch**:
 - `http://site.com` and `https://site.com` (different *scheme*).
 - `http://site.com` and `http://sub.site.com` (different *host*).
 - `http://site.com:80` and `http://site.com:8080` (different *port*).

3.1.3 SOP Relaxation (`document.domain`)

Subdomains wishing to cooperate (e.g., `catalog.mystore.com` and `orders.mystore.com`) can explicitly relax the SOP by setting `document.domain` to their common suffix (e.g., `mystore.com`). This makes their DOM objects mutually accessible, but also exposes them to **all** other subdomains of that suffix.

3.2 Same-Origin Policy for Cookies

The cookie policy is **different** from the DOM SOP.

- **Port Excluded:** A cookie returned to a host (server) is available to **all ports** on that host; the port is excluded from the cookie's origin.
- **Adjustable Scope:** The `Domain` and `Path` attributes allow the server to **broaden or restrict** the cookie's scope, respectively, to a **suffix domain** or a **prefixed URI path**.

Example:

- If a cookie is set by the server `products.mystore.com` with the attribute `Domain=mystore.com`, the browser will send that cookie to subdomains such as `payments.mystore.com` or `blog.mystore.com` as well (domain broadening).
- If a cookie is set with the attribute `Path=/cart`, the browser will only send it with requests that begin with that URI path (for example, `/cart/checkout.html`), excluding all other sections of the site like `/blog` (path restriction).

Cookie control is based on the path (`Path`), whereas JavaScript access to HTTP cookies is **not** restricted by the path.

3.3 SOP and AJAX

The SOP also applies to asynchronous communications between browser and server facilitated by **AJAX** (Asynchronous JavaScript and XML) and the `XMLHttpRequest` object. Asynchronous HTTP requests by scripts are restricted to using the **same origin server** that provided the base document in which the script is embedded.

4 Authentication Cookies, Malicious Scripts, and CSRF

4.1 Session ID and Cookie Theft

Servers utilize a randomly chosen **Session ID** stored in an HTTP cookie to maintain the state of an interaction (session). Once a user logs in, the Session ID becomes an **Authentication Cookie**, which authorizes subsequent requests without requiring re-authentication.

4.1.1 Security Risks (Cookie Theft)

The theft of authentication cookies can lead to session hijacking and unauthorized access to sensitive resources. The primary vectors of theft include:

- **Malicious JavaScript** (e.g., XSS attacks): If the client inadvertently executes an injected malicious script on a legitimate page, it can send the contents of `document.cookie` to a malicious site.
- **HTTP Proxies** or Man-in-the-Middle attacks: If the cookie is transmitted over unencrypted connections (cleartext HTTP).
- **Client-side malware** that accesses memory or the filesystem.

4.1.2 Cookie Protection

The server must enforce security measures on cookies:

- **HttpOnly**: Prevents cookie access via JavaScript (XSS mitigation).
- **Secure**: Forces the cookie to be sent only over encrypted connections (HTTPS/TLS).
- **Integrity and Confidentiality**: The server should **encrypt** and protect the **integrity (MAC)** of cookies containing sensitive data before sending them to the client, preventing client-side tampering.

4.2 Cross-Site Request Forgery (CSRF)

Cross-Site Request Forgery (CSRF) (or Session Riding) is an attack that exploits the trust a website places in the user's browser. If a user is currently logged in (i.e., possesses an Authentication Cookie), a **pre-authorized** HTTP request generated by the victim's browser (even if initiated by a malicious site) will be executed by the target server. Because trust is placed in the browser, if an attacker forces a logged-in user's browser to submit a request to a bank, the attack succeeds. The attacker never needs to know the content of the cookie itself.

4.2.1 CSRF Attack Vectors

The attacker induces the victim to send a harmful HTTP request to the target site, leveraging HTML elements that trigger automatic requests:

- **Links** (`<a href="...">`): Requires action (a click) by the victim.
- **Image Tags** (``): The most common element; the browser automatically attempts to load the image, triggering a GET request to the malicious URL without the victim noticing (often hidden with a 0×0 pixel size).
- **Hidden Forms** with event handlers (`onload`): To automatically generate POST requests.

4.2.2 CSRF Mitigation

The primary defense against CSRF is the use of **Secret Validation Tokens (Anti-CSRF Tokens)**.

- At the start of a session, the server sends a unique secret token to the browser.
- Every subsequent HTTP request that modifies state on the server (POST, PUT, etc.) must include this token (ensuring an external request lacks unauthorized capabilities).
- A CSRF attacker, lacking access to this token (as it is not automatically sent with cookies), cannot construct the forged request.

5 Cross-Site Scripting (XSS)

Cross-Site Scripting (XSS) is a broad class of attacks based on the injection of malicious HTML tags or scripts into web pages. This results in the execution of unintended JavaScript code within the victim's browser, violating the spirit of the SOP.

5.0.1 Potential Impact of XSS

The execution of injected scripts can lead to:

1. Access and **theft of Authentication Cookies** and session tokens (the classic case).
2. **Redirection** of the browser to attacker-controlled sites (phishing).
3. Accessing or modifying **DOM data** and data stored in the browser for the current site.
4. **Rewriting the document** displayed to the client (`document.write()`).

5.1 Types of XSS Attacks

5.1.1 Stored XSS (Persistent)

- The malicious code is **permanently stored** on the server (e.g., in a database or filesystem), such as inside a forum comment or a blog post. If unfiltered, it enters the database exactly as written.
- When any visitor loads the page containing the stored content, the script is executed within the origin context of the legitimate site because it is interpreted as code.

5.1.2 Reflected XSS (Non-Persistent)

- The malicious script is not stored, but is contained within the request URL.
- A vulnerable site **reflects** un-sanitized user input (e.g., a URL parameter) back into the response page (e.g., an error page: "File not found: *injected-filename*").
- The victim's browser executes the reflected script. The attacker engineers the victim to click a link containing the script (for example, by sending a legitimate link appended with malicious script via email); once clicked, the malicious code executes.

5.1.3 DOM-based XSS

- The injection and execution occur entirely **client-side**.
- The vulnerable code manipulates the browser's DOM environment based on untrusted source data (e.g., a URL fragment, `document.location`), without direct server interaction.

Example of DOM-based XSS

This attack exploits the manipulation of the DOM **client-side** by JavaScript code that processes **untrusted data** (frequently from the URL fragment input).

- **Vulnerable Code (Client-Side):** A poorly structured but legitimate piece of JavaScript reads input from the URL fragment (`document.location.hash`) and writes it directly to the page's HTML, for instance using `document.write()`:

```
document.write("Welcome: " + document.location.hash.substring(1));
```

- **Attack Vector (Forged URL):** The attacker tricks the victim into loading a URL containing malicious code within the fragment, which is **not** transmitted to the server:

```
http://site.com/page.html#<script>alert('XSS')</script>
```

- **Execution (Local):**

1. The browser loads the page; the server **does not see** the fragment (`#...`).
2. The vulnerable JavaScript reads the fragment and inserts it into the HTML as code.
3. The script `<script>alert('XSS')</script>` executes within the origin context (`site.com`).

Conclusion: The server is bypassed entirely; the vulnerability resides exclusively in the logic of the client-side JavaScript code that implicitly trusts local input.

5.2 XSS Mitigation and Input Sanitization

The P15 principle (**DATA-TYPE-VERIFICATION**) is fundamental here. Defense relies on **Input Sanitization**, the process of stripping potentially malicious components from user-supplied data.

- **Output Escaping (Filter Evasion):** Replacing special characters like `<` and `>` with their corresponding HTML entities (e.g., `<` and `>`). The browser displays them as literal text instead of interpreting them as markup syntax.
- **Canonicalization:** Mapping input to a single, standard encoding format to neutralize evasion tactics such as **alternative encodings** (e.g., using `<` instead of `<`).
- **Whitelisting (Positive Validation):** The preferred approach. Allowing only "known-good" input and expected structures (e.g., if a username is expected, reject any script syntax like `<script>`).

6 SQL Injection

SQL Injection exploits the practice of dynamically constructing SQL queries server-side using unsanitized user input (e.g., from forms, cookies, or parameters).

6.1 Attack Mechanism

The attacker submits data that **alters the syntactic structure** of the SQL query, appending custom commands.

- **Exploiting Single Quotes:** By entering a single quote (`'`), the attacker prematurely terminates an open SQL string literal within the server's code, enabling the introduction of custom commands.
- **Comments:** Using `-` (in certain SQL dialects) comments out the remainder of the original query, effectively neutralizing safety checks (such as password verification).
- **Logical Conditions:** Injecting a tautology (a condition that is always true, e.g., `' OR 1=1 -`) forces the query to execute across all records.
- **Command Separators:** Using a semicolon (`;`) can allow the execution of a separate, arbitrary SQL statement altogether (e.g., `DROP TABLE pswdtab -`).

6.2 SQL Injection Mitigation

Defense focuses on establishing a clean separation between the SQL code structure and input data values.

- **Prepared Statements (Parameterized Queries):** The primary defense mechanism. The SQL query structure is sent to the database separately from the input parameters. The database distinguishes code from data unambiguously, ensuring that input is processed strictly as a literal value and never as an executable command.
- **Stored Procedures:** Similar to Prepared Statements, these store the query logic on the database server itself and accept parameters, preventing structural injection.
- **Input Validation (Whitelisting):** Verification that input strictly conforms to expected type and format constraints (e.g., ensuring an ID field contains only digits).
- **Escaping:** Replacing special characters like single quotes with their literal string representations (e.g., `"` or `\'`). This is a reactive approach and is not sufficient on its own.